From Suspicious Login to Recovery

A step-by-step SOC portfolio walkthrough

Oluwaseyi Falode
KQL investigation | Jira case ownership | Python detection and response

Three failed passwords followed by a successful login: a simple sequence with two very different explanations. This project teaches you how to investigate the records, document uncertainty, generate an alert, and verify that response controls actually work.

This is a build-along manual. Start with a browser, follow the numbered actions, run the included commands, and compare your outputs with the checkpoints. Complete code is included in Appendix A; the companion ZIP makes copying easier but is not required to understand the procedure.

Training scope. All users, IP examples, orders and events are fictional. Azure and Jira hold the practice investigation. A separate Python application demonstrates working containment and recovery. No enterprise identity controls or real customer systems are changed.

How screenshots are used. Original execution and setup screenshots illustrate the steps they actually show. Some code and expected outputs are written examples where no screenshot was captured. Cropped figures remove unrelated browser and personal content; they do not fabricate results.

Version 2 | 24 September 2026 | Beginner-friendly browser workflow

Read this before you start

You need	Purpose
Mac or another computer + browser	No local virtual machine required. Mac shortcuts are used below.
Personal Microsoft account	Azure Data Explorer free cluster. Use the free-cluster route.
Personal Atlassian account	Jira Free case tracking. A standard task is enough.
Personal Google account	Google Colab with a standard CPU runtime.
A folder named SOC-Project-1	Save screenshots, queries, your notebook and exports.

The free Azure Data Explorer route does not require an Azure subscription or credit card [1]. Colab offers free access with variable resource availability [2]. Select Jira Free [3]. A paid trial, GPU or marketplace extension is not required for this lab. If a screen asks you to upgrade, return to the free route rather than purchasing anything.

The two environments are separate. KQL queries run in Azure Data Explorer. Python cells run in Colab. The notebook does not read your Azure database, change Jira, or administer Okta. It recreates a small event pattern and application locally inside its runtime.

Editor rule: in Azure, click inside the query editor, press Command+A, replace the text with the next block, then select Run. Clearing editor text does not delete stored tables. In Colab, keep earlier cells and add/run later cells; they share variables.

Pace: use the checkpoints, not a stopwatch. You can pause after Azure or Jira. Complete the Python section in one runtime session, or restart its cells from the beginning.

Your route through the lab

Part	What you will do
Steps 1-3 / pages 4-6	Create the free cluster, verify the editor and define a table.
Steps 4-7 / pages 7-10	Load six login events, investigate Alex and review application activity.
Steps 8-11 / pages 11-14	Create a Jira case, link follow-up work and record evidence.
Steps 12-16 / pages 15-19	Run a detector, ten checks, an alert worker and a training application.
Steps 17-20 / pages 20-23	Verify recovery, assess a benign scenario, close the lab and save evidence.
Appendix A / pages 24-32	Complete Python code, split into six executable Colab cells.
Appendices B-C / pages 33-35	Troubleshooting, screenshot provenance and official references.

Terms in plain English

Cluster: the service hosting your data. **Database:** a container for tables. **Table:** rows of events with named columns. **Telemetry:** records describing activity. **Query:** a question expressed in code. **Alert:** a rule match needing attention. **Incident:** activity that warrants coordinated investigation or response.

Session: an authenticated period of access. A **token** lets the application recognise that session. **MFA** means more than one authentication factor. **Containment** restricts ongoing access; **recovery** restores intended access after the relevant checks.

The finished evidence should let another person answer three questions: What did you observe? What did you do? How did you check the result?

Step 1 - Create the free workspace

1. Open dataexplorer.azure.com/publicfreecluster and sign in with your Microsoft account.

2. Select My cluster, then Create cluster if you do not already have a free cluster.

3. Enter **soc-practice-lab** as the display name and **SOCTraining** as the database name. Your cluster name may differ from mine.

4. Use Europe if that is the offered location. These records contain no real personal data. Review the terms, tick the agreement box and select Create.

5. Wait for the overview. If you already have the cluster but no empty training database, use Create database to make **SOCWalkthrough** and use that database throughout instead.

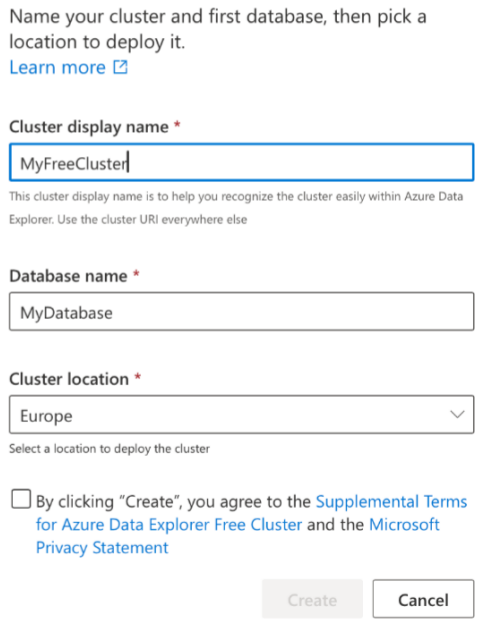


Figure 1. Creation form before submission. It still shows default names; replace them with your chosen lab names. Source: uploaded lab screenshot; relevant region shown.

Checkpoint: You can open your cluster and see the selected database. Do not create a paid Azure resource group or upgrade the cluster for this exercise.

Step 2 - Run your first KQL query

1. Open Query or Query data. Select your training database in the database selector above the editor.

2. Click inside the editor. Press Command+A and delete any starter text. Paste the block below.

3. Select the blue Run button. Read the results grid beneath the editor.

```
print Message = "My SOC lab is ready"
```

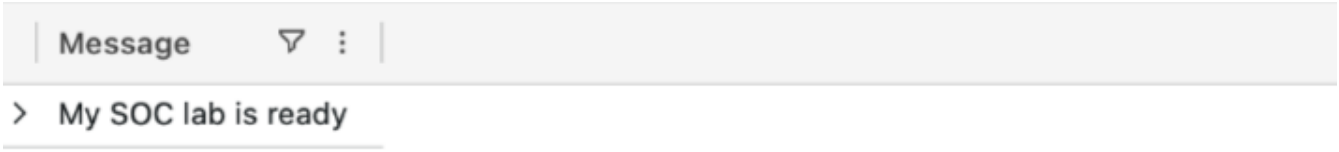


Figure 2. A one-row result confirms the query ran. Source: uploaded lab screenshot; relevant region shown.

Why this works: print creates a result directly; it does not need a table. Message becomes the column name, and the quoted text becomes its value. Kusto is the technology; KQL is the language you are typing.

Checkpoint: One row under Message says My SOC lab is ready. Keep a screenshot. For the next step, replace this editor text; you do not need to keep the print command above it.

If it fails: use straight quotation marks, verify the database selector, and copy only the command, not the surrounding explanation.

Step 3 - Create the login table

A schema defines the column names and data types. TimeGenerated holds a timestamp; the remaining fields are text. Use an empty personal training database so this exercise does not conflict with old data.

1. Replace the print command with the table-creation command. Select Run once.

2. Replace that command with LoginLogs on its own, then Run to inspect the table.

```
.create table LoginLogs (
    TimeGenerated:datetime, UserName:string,
    IPAddress:string, Result:string, Application:string
)

LoginLogs
```

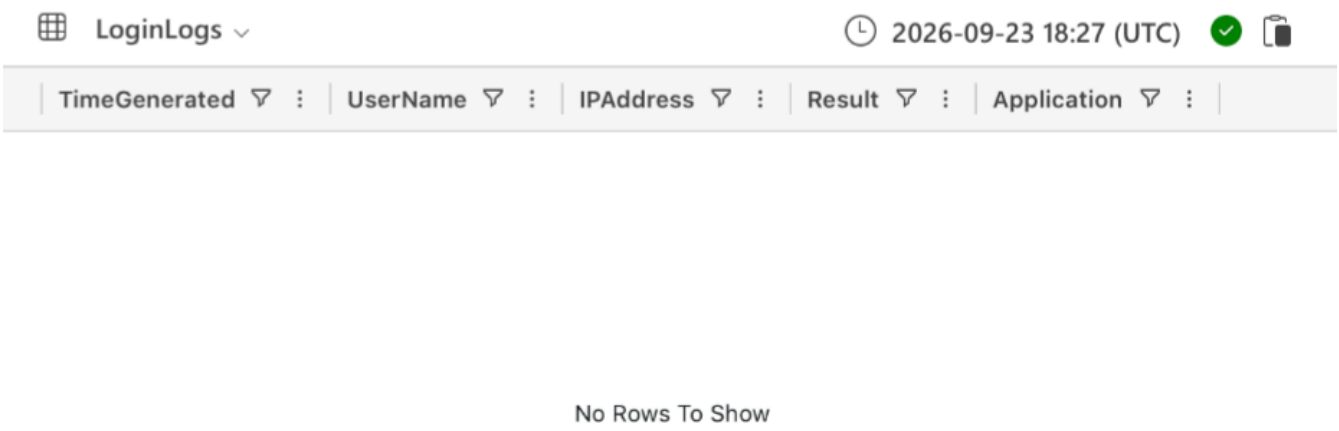


Figure 3. The empty table shows its five columns and no rows. Source: uploaded lab screenshot; relevant region shown.

Checkpoint: An empty result is correct before ingestion. If the table already exists, inspect it first. Do not repeatedly recreate it or delete an existing database to resolve an error.

Step 4 - Load the six login events

Ingest means load data into the table. The following rows use a fixed date so every reader can reproduce the same scenario. The Z suffix means UTC. Do not replace the date with today or use a last-24-hours filter later.

1. Replace the editor text with this entire ingestion block, including all six records. Select Run once.

2. After it finishes, replace the editor text with the count query below and Run.

```
.ingest inline into table LoginLogs <|
2026-09-23T09:00:00Z,alex,192.0.2.10,Failed,DeliveryPortal
2026-09-23T09:01:00Z,alex,192.0.2.10,Failed,DeliveryPortal
2026-09-23T09:02:00Z,alex,192.0.2.10,Failed,DeliveryPortal
2026-09-23T09:03:00Z,alex,192.0.2.10,Success,DeliveryPortal
2026-09-23T09:04:00Z,maya,198.51.100.20,Success,DeliveryPortal
2026-09-23T09:05:00Z,ben,203.0.113.30,Failed,DeliveryPortal

LoginLogs | count
```

ExtentId	ItemLoaded	Duration	HasErrors	OperationId
c74d0419-1df7-47ef-9	inproc:c9cb7fb0-462l	00:00:00.2849599	false	249f72ee-badf-4000-

Figure 4. Historical ingestion confirmation, with HasErrors = false. That screenshot loaded one row; the consolidated block above loads all six together. Source: uploaded lab screenshot; relevant region shown.

Checkpoint: The count query returns 6. If it temporarily returns 0, rerun the read-only count query after ingestion completes. Do not resend the ingestion block: it appends duplicate rows. If count is above 6, use the troubleshooting appendix before continuing.

Step 5 - Investigate Alex

1. Replace the editor text with the timeline query below. Run it.

2. Read the records from top to bottom: failed attempts at 09:00, 09:01 and 09:02; success at 09:03.

3. Next run the failure-count query. The two queries are separate editor operations.

```
LoginLogs
| where UserName == "alex"
| order by TimeGenerated asc

LoginLogs
| where UserName == "alex"
| where Result == "Failed"
| summarize FailedAttempts = count()
```

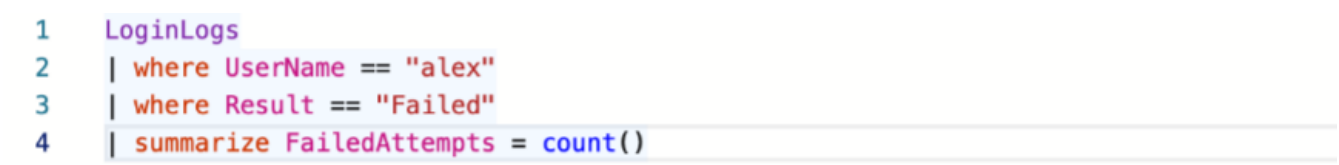


Figure 5. Filtering and counting the failed events. Source: uploaded lab screenshot; relevant region shown.

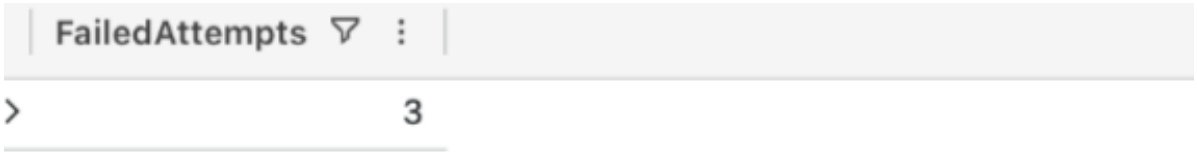


Figure 6. The recorded failure count is three. Source: uploaded lab screenshot; relevant region shown.

Read the syntax: the pipe | passes results to the next instruction. where filters rows; == means equals; order by ... asc shows oldest first; summarize combines rows; count() counts them.

Checkpoint: You can explain the sequence and the count. Neither proves who typed the password. Write “suspicious pattern requiring investigation,” not “confirmed attacker.”

Step 6 - Understand the first detection rule

Now find accounts with at least three failures and one success in the fixed review interval. This is an investigation query; pressing Run does not deploy an always-on alert service.

```
LoginLogs
| where TimeGenerated >= datetime(2026-09-23T09:00:00Z)
| where TimeGenerated < datetime(2026-09-23T09:10:00Z)
| summarize
    Failures = countif(Result == "Failed"),
    Successes = countif(Result == "Success")
  by UserName, IPAddress, Application
| where Failures >= 3 and Successes >= 1
```

Expected match	Value
UserName / IPAddress	alex / 192.0.2.10
Application	DeliveryPortal
Failures / Successes	3 / 1

Threshold: the numerical condition that triggers a match. **Fidelity:** how reliably a detection identifies activity worth investigating. More alerts do not necessarily mean better coverage.

Important limitation: this query checks counts inside an interval. It does not require success to occur after the failures. The chronological query in Step 5 shows the order for this case; the Python detector later makes order part of the rule itself.

Try a pivot: run the next query to inspect every login from the suspicious IP. An IP can be shared, so it is not proof of a single person.

```
LoginLogs
| where IPAddress == "192.0.2.10"
| order by TimeGenerated asc
```

Checkpoint: The summary query returns one match; the IP pivot returns Alex's four events. No screenshot of this summary result was captured; the values above are expected fixture results.

Step 7 - Follow the application activity

1. Create ApplicationLogs with the first command. Run it separately.

2. Replace the editor with the ingestion block. Run once.

3. Replace the editor with the final query to read Alex's actions in time order.

```
.create table ApplicationLogs (
    TimeGenerated:datetime, UserName:string,
    Application:string, Action:string,
    Resource:string, Result:string
)

.ingest inline into table ApplicationLogs <|
2026-09-23T09:04:00Z,alex,DeliveryPortal,ViewOrder,Order-1042,Success
2026-09-23T09:05:00Z,alex,DeliveryPortal,ExportCustomerData,CustomerDirectory,Denied
2026-09-23T09:06:00Z,alex,DeliveryPortal,Logout,Session,Success

ApplicationLogs
| where UserName == "alex"
| order by TimeGenerated asc
```

Expected result: three rows: a successful order view, a denied customer-directory export and a logout. The code is a reproducible fixture; there is no separate screenshot of this result in the supplied evidence.

What you can conclude: the recorded export attempt was denied. **What you cannot conclude:** no data was exposed, no other actions occurred, or every session was terminated. The two tables lack a shared session ID; user and time alone do not prove session attribution.

Checkpoint: Record both the observed activity and the uncertainty. Password guessing, reuse or phishing remain hypotheses, not established causes.

Step 8 - Create your case workspace

1. Open atlassian.com/software/jira/free and create or open your personal Jira Free site.

2. From Spaces or Projects, choose Create space/project. Select a basic Kanban template for this exercise. Labels vary with Jira versions.

3. Name it **SOC Investigations**. If asked for a management style, use team-managed. Set the key to SOC if available. Complete the creation flow; no teammates need to be invited.

4. Open its Board or List. You should have To Do, In Progress and Done states.

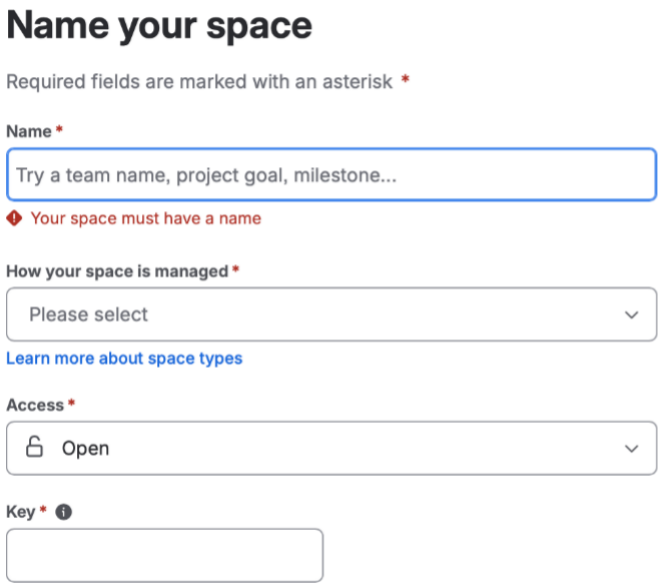


Figure 7. The space name and key fields before completion. Source: uploaded lab screenshot; relevant region shown.

A Jira space is your work area. A work item is an individual record. For this lab, a Task can serve as an incident case. A paid incident-management product is not required.

Checkpoint: You are in your own SOC Investigations space. Your site URL and item numbers will differ from mine. Keep real credentials and personal customer data out of this public practice case.

Step 9 - Open and own the incident case

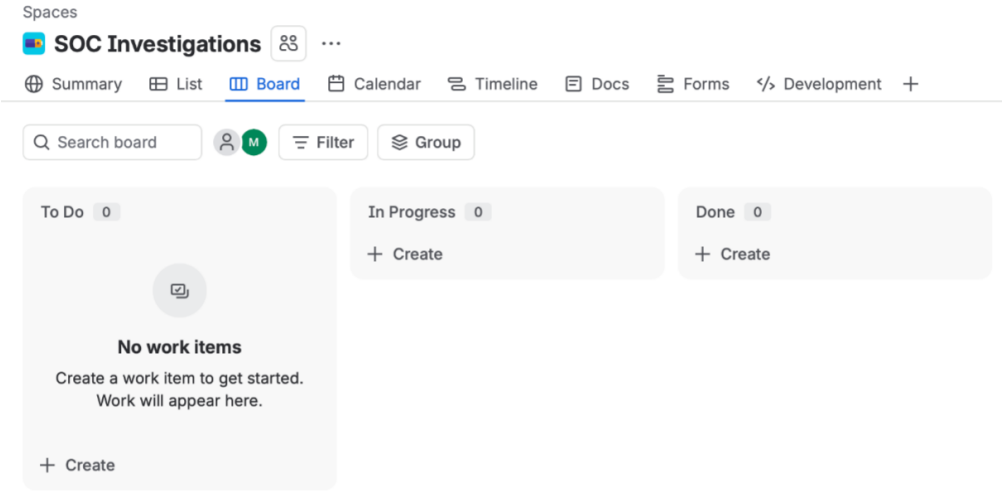


Figure 8. The empty SOC board before case creation. Source: uploaded lab screenshot; relevant region shown.

1. Choose Create. Select your SOC space and the Task work type.

2. Use the summary **[LAB] Alex account - repeated failures and successful access**. Paste the description below.

3. Set Assignee to yourself. Save the work item, open it and move its status to In Progress. Record its generated key. My main case was SOC-2; yours may be SOC-1 or another number.

Description to paste

Scope: fictional DeliveryPortal identity investigation. Alex has three failed logins followed by success from 192.0.2.10 at 09:00-09:03 UTC on 23 September 2026. Application records show an order view and a denied customer export. Goal: assess the activity, document uncertainty and practise response verification in a separate training application. Owner: me. No live enterprise changes are authorised by this lab.

Checkpoint: The case has an accountable owner and an In Progress status. “Owning end to end” means tracking evidence, decisions and follow-up; assignment is how that ownership is recorded.

Step 10 - Link the investigation follow-up

1. Create another Task titled **[LAB] Investigate Alex access method**. Assign it to yourself. Describe the question: what evidence explains authentication, and what remains unknown?

2. Open the follow-up item. Find Linked work items, Link work item, or the item's ... menu. Choose the linking control, not Share and not Add subtask.

3. Select **relates to**. Search for your main case key, select it and save the link.

4. Verify the saved relationship shows the main case. A blank link-entry row below it is not another saved relationship.

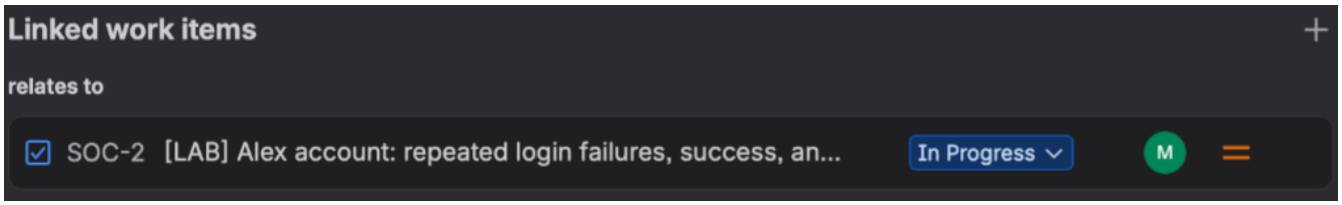


Figure 9. The saved relates-to relationship. This screenshot shows an intermediate case state. Source: uploaded lab screenshot; relevant region shown.

Why this relationship? “Relates to” records context. “Is blocked by” means progress depends on another item. Choose the relationship that reflects reality instead of leaving the default selected.

Common mistake: entering a case key into Add subtask creates a new child item instead of linking an existing item. If you do this in your own lab, mark the accidental item as created in error and resolve it; then create the correct link. Do not fabricate investigation work for the accidental item.

Checkpoint: You can navigate from follow-up to main case. No additional application is needed for linking or comments.

Step 11 - Record evidence and its limits

1. Open the follow-up case. Scroll to Activity and select Comments. Click Add a comment.

2. Paste the short note below. Choose Save. Reopen or scroll to confirm the note appears in the activity history.

3. Return to the main case and add a concise update linking to the follow-up.

Comment to paste

[LAB] LoginLogs shows three failures followed by success for Alex. ApplicationLogs shows an order view and a denied export. The source tables do not share a session identifier. Password acquisition and full exposure scope remain undetermined. Next: evaluate the supplied authentication scenario and complete response verification in the isolated training app.

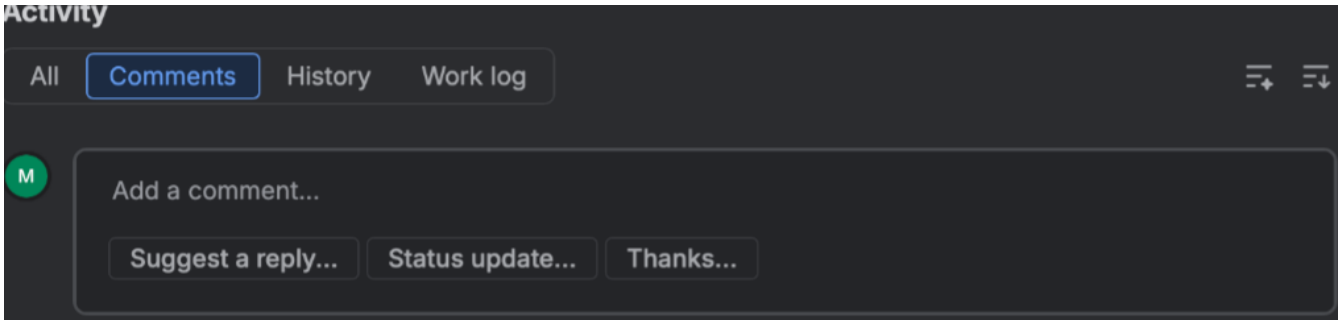


Figure 10. Activity, Comments and the comment entry area. Source: uploaded lab screenshot; relevant region shown.

Optional tabletop context from the original exercise: a supplied authentication record described an accepted password, an expired MFA policy exception and LAB-SESSION-001. Those are fictional scenario facts, not logs collected from an identity provider. You can discuss their implications, but do not claim to have inspected or removed a real policy.

Checkpoint: A useful note separates observation, source, uncertainty and next action. Keep operational updates short; attach detailed evidence rather than repeatedly pasting the whole case history.

Step 12 - Set up the Python notebook

1. Open colab.research.google.com and sign in with your personal Google account.

2. PDF-only route: choose New notebook and rename it SOC-Project-1. Use + Code to add six code cells. Copy the complete Cells 1-6 from Appendix A, in order. If one cell spans pages, all its pages belong in the same code cell.

3. Companion route: choose File > Upload notebook and upload **Project1-Walkthrough.ipynb** from the ZIP. It contains the same six code cells. Choose either route, not both.

4. Use the standard Python CPU runtime. Click Connect if necessary. No package installation, Drive mount, GPU or API key is required by this code.

5. Run one cell at a time using the triangle beside it. Keep earlier cells; later cells use their variables.

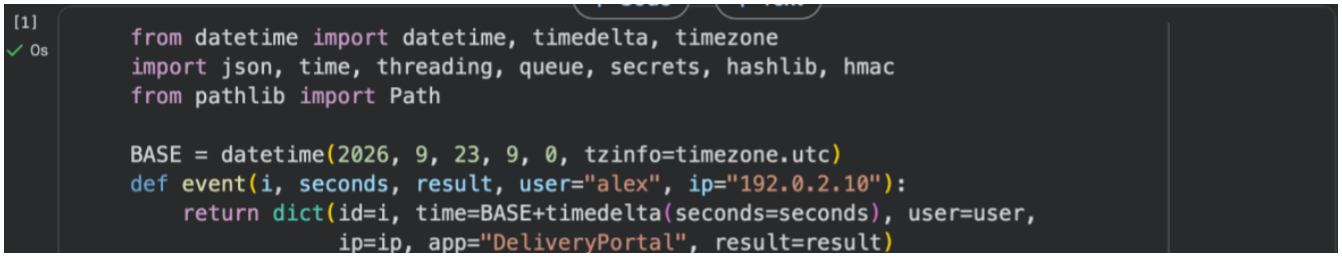


Figure 11. The original notebook after the first setup cell ran. The companion uses the same logic with clearer formatting. Source: uploaded lab screenshot; relevant region shown.

Checkpoint: The notebook is open and you know how to add and run a code cell. Python belongs here; Azure management commands beginning with a dot do not.

Step 13 - Test the detector

1. Read and run Appendix Cell 1. It imports standard libraries, creates fictional events and defines detect().

2. Run Appendix Cell 2 to check ten scenarios. Do not continue if an assertion fails.

3. Capture the output and explain why the legitimate repeated-typo example intentionally produces an alert.

Inside the rule: first remove duplicate event IDs and sort records. For each success, find failures from the same user, IP and application. Require at least three failures within the preceding five minutes, strictly earlier than the success. The exact five-minute lower boundary is included.

Check	Expected alerts
Failures then success / exact five-minute boundary	1 / 1
One typo / no success / success first	0 / 0 / 0
Outside window / duplicate inflation	0 / 0
Equal timestamps / another IP	0 / 0
Legitimate repeated typos with the same pattern	1

Expected final line: 10 checks passed. A matching pattern still needs investigation. This is an expected code result, not a screenshot from the original run.

The tests ask whether the detector behaves as designed. They do not establish production precision or recall. Event IDs must be unique and trustworthy; reusing an ID for different records can overwrite a row in this simplified implementation.

Checkpoint: You understand the difference between a correct pattern match and confirmed malicious activity. Raising a threshold to hide one benign example is not a substitute for context.

Step 14 – Emit an automatic replay alert

1. Run Appendix Cell 3 after Cells 1 and 2. It starts a worker and places four fictional events into a queue, then repeats the successful event.

2. Wait for the worker to stop. Read the alert and its evidence_ids.

3. Check that exactly one alert was emitted and that the file project1_alerts.jsonl appears in the runtime file browser.



```
AUTOMATIC ALERT: alex — 3 failures before success
PASS: one alert, repeated delivery suppressed, worker stopped.
{
  "alert_id": "rule1:s1",
  "user": "alex",
  "ip": "192.0.2.10",
  "application": "DeliveryPortal",
  "failures": 3,
  "success_time": "2026-09-23T09:03:00+00:00",
  "evidence_ids": [
    "f1",
    "f2",
    "f3",
    "s1"
  ],
  "disposition": "Needs analyst review"
}
```

Figure 12. One automatic alert; repeated delivery suppressed; worker stopped. Source: uploaded lab screenshot; relevant region shown.

What the code adds: a queue carries incoming events; a worker applies the rule; a set remembers emitted alert IDs. Each incoming event is processed without manually running a new query. JSONL stores one JSON object per line.

Checkpoint: One alert, not two. Its disposition is Needs analyst review. This is a short synthetic replay, not a scheduled or 24/7 SIEM service. The in-memory state disappears when the runtime resets.

Step 15 – Start the training application

1. Run Appendix Cell 4 once. It creates the isolated HTTP server, the fictional Alex account, a password hash and an empty session store.

2. The code logs in and accesses a fictional order before changing controls. Read the baseline output.

3. Keep the runtime connected. Do not open the server URL in Safari: 127.0.0.1 refers to the Colab runtime, not your Mac. The notebook client sends the requests.

Expected: BEFORE containment: login 200; protected order access 200.

How it works: /login accepts the disposable password and returns a session token. /order checks that token before returning a fictional order. The code stores a salted password hash rather than the plain password as the account credential.

Code or term	Meaning
127.0.0.1	Loopback: access stays inside the runtime.
200	The tested request succeeded.
401	The presented authentication was rejected.
403	The app forbids access, such as for a disabled account.

Checkpoint: Both baseline requests return 200. Without a working baseline, a later failure could be a broken setup rather than successful containment.

Use only the disposable passwords in the appendix. This app deliberately omits real MFA, enterprise permissions, session expiry and hardened admin APIs. The control actions are Python operations for learning cause and effect.

Step 16 - Contain and verify

Containment means reducing the ability to continue unwanted activity. In this training app, you can test the effect directly.

1. Before running Appendix Cell 5, predict the old-session result and the fresh-login result.

2. Run Cell 5. It clears sessions and sets the account's enabled flag to False.

3. The same cell requests the order with the old token and attempts a new login with the old password. Read both results.

```
sessions.clear()
record("admin-revoke-sessions","completed")
account["enabled"]=False
record("admin-disable-account","completed")
old_access=request("/order",token=old_token)[0]
fresh_login=request("/login",{"user":"alex","password":OLD_PASSWORD})[0]
assert old_access==401 and fresh_login==403
print("CONTAINED: old session",old_access,"; fresh login",fresh_login)
print("Verified by rejected requests, not merely by an admin success message.")

CONTAINED: old session 401 ; fresh login 403
Verified by rejected requests, not merely by an admin success message.
```

Figure 13. Containment code and the rejected requests. Source: uploaded lab screenshot; relevant region shown.

Checkpoint: Old session = 401; fresh login = 403. The assertions stop execution if these values are wrong.

Why two checks? Revoking a session addresses existing access. Disabling the account prevents a fresh login. Testing only the admin command would not prove that either access path was blocked.

Case update after your own run: [LAB] In the isolated training app, session revocation and account disablement completed. Verification: old-token order request returned 401; fresh login returned 403. Enterprise accounts were not changed.

Step 17 – Recover and verify again

1. Run Appendix Cell 6. It sets a new disposable password hash and re-enables the account.

2. Read the assertions: old password rejected, old token rejected, new login accepted and new-session order access accepted.

3. Confirm that project1_response_audit.json was written and that the training server stopped.

```
RECOVERY PASS: old password 401; old token 401; new login and order access 200.
Training server stopped. No enterprise accounts were changed.
```

Figure 14. Recovery verification and server shutdown. Source: uploaded lab screenshot; relevant region shown.

Expected: old password 401; old token 401; new login and order access 200. Recovery is complete for the paths this training application tests.

What would precede real recovery? Appropriate identity verification, cause assessment, device checks, MFA/recovery-method review, relevant remediation and approval under the organisation's process. A password reset alone does not resolve every type of compromise.

Checkpoint: The old paths remain blocked and intended new access works. These results apply to this separate application; they do not prove that the fictional Azure/Jira case was remediated in an enterprise system.

If you want to repeat the response exercise: rerun Cell 4 to initialise a new app and baseline, then Cells 5 and 6. Do not run Cell 5 after shutdown without Cell 4.

Step 18 – Decide whether an alert is benign

Taylor scenario: three failed logins at 11:00, 11:01 and 11:02 are followed by success at 11:03 from the same user/IP/application. The exercise states that a verified callback, known managed device, expected MFA and matching authorised order session support legitimate activity.

Pause and answer: Does the rule match? Is malicious activity established? Would you automatically disable the account? Which records would you inspect?

Question	How to reason
Will it alert?	Yes, assuming distinct events: three failures precede success within five minutes.
Is it confirmed malicious?	No. The pattern alone cannot decide. Verified context could support benign activity.
What evidence supports the user?	Exact authentication details, corresponding device-management record and session-linked application audit.
Should I disable automatically?	Not from this pattern alone in the stated benign context. Follow response policy if other risk appears.

Do not invent the verification. No Taylor call, company directory lookup, device-inventory check or MFA inspection occurred in this lab. Those facts are scenario assumptions. “Legitimate if corroborated” is the defensible conditional conclusion.

For a real check, use an approved pre-existing contact route, follow the organisation's identity-verification procedure, and document it. Match the sign-in device ID to inventory. Inspect the specific authentication event: a successful password login does not establish MFA, and an existing token claim differs from a fresh challenge [6].

Checkpoint: A benign alert is not necessarily a broken rule. The rule can correctly match events while an analyst determines that the activity was authorised. Record review period and telemetry gaps.

Step 19 – Close the lab with evidence

1. Open the main Jira case and review its findings and linked follow-up.

2. Add the completion note below only after your own outputs match. Replace the test-result statement if any check failed.

3. Attach your screenshots or exported notebook where appropriate. Resolve the follow-up when its question and limitations are documented.

4. Move the main lab case to Done only when the practical checks and notes are complete. If not, keep it In Progress and record the blocker.

Completion note template

[LAB] Investigation identified repeated failures followed by successful access in the fictional data. Application records show an order view and a denied export; password acquisition and full exposure remain undetermined. The detector completed ten checks, and the replay worker emitted one alert with duplicate suppression. In the separate training application, old-session access returned 401 and disabled-account login returned 403. Recovery rejected the old password/token and accepted new access with 200 responses. Exercise complete with documented limitations; no enterprise controls were changed.

Closure is a decision, not just a status. In production, severity, evidence preservation, approval, residual risk, monitoring and handover follow the organisation's requirements. This lab closure does not establish those conditions for a real incident.

What the original screenshots establish: query execution, case work and selected notebook results. They do not show final closure of the main Jira case. Your own final screenshot should show the state you actually reached.

Checkpoint: Another reader can distinguish the synthetic investigation from the working training-app controls, and can see what remains unknown.

Step 20 – Save a reproducible portfolio

1. In Colab, choose File > Download > Download .ipynb to save your executed notebook.

2. Open the folder/files panel on the left. Refresh if needed. Use each file's menu to download project1_alerts.jsonl and project1_response_audit.json. Do this before the temporary runtime ends.

3. Save your Azure query text and capture the key results: six rows loaded, Alex's sequence, failure count and application actions.

4. Capture the Jira case description, correct link and final update. Use a window or region screenshot that excludes unrelated tabs and personal information. On a Mac, Shift+Command+4 lets you select a region.

5. Keep the PDF, notebook, code and selected evidence together. Explain the training scope when you publish.

Portfolio file	What it lets another person do
Project1-SOC-Walkthrough.pdf	Follow the full build without needing this conversation.
Project1-Walkthrough.ipynb	Execute the six Python cells in Colab.
code/azure-queries.kql	Copy the numbered query/command blocks one at a time.
code/cell-01.py through cell-06.py	Recreate a notebook from plain code.
screenshots/	See the selected original screen regions.
Your own saved outputs	Review your execution results and timestamps.

Cleanup: Cell 6 stops the training server. After exporting, disconnect the Colab runtime. Keep the free Azure lab if you want further practice; deleting a cluster/database removes its stored lab data, so export first and only delete resources you deliberately created for this exercise.

You have now practised the full learning workflow: build, ingest, query, investigate, document, detect, test, contain, recover and explain.

Appendix A - Complete executable Python

This appendix contains the full six-cell implementation. You can reproduce the Python exercise using only this PDF and a blank Colab notebook. Copy code in order; retain indentation. A cell may span pages. Continue copying until the next cell heading. Do not include page headers or footers.

The companion .py files contain exactly the same code. Their only dependencies are Python standard-library modules. The notebook has no stored outputs so each reader generates a fresh run. The code was reformatted for readability without changing its behaviour.

Cell map: 1 defines events/detection/tests; 2 runs tests; 3 replays events; 4 starts the application; 5 contains and verifies; 6 recovers, exports and stops.

Python Cell 1 - Define events and detection tests

Paste the entire following block into one code cell. Continue on the next page if the code spans pages. Run only after earlier cells have completed.

```
from datetime import datetime, timedelta, timezone
import json, time, threading, queue, secrets, hashlib, hmac
from pathlib import Path

BASE = datetime(2026, 9, 23, 9, 0, tzinfo=timezone.utc)


def event(i, seconds, result, user="alex", ip="192.0.2.10"):
    return dict(
        id=i,
        time=BASE + timedelta(seconds=seconds),
        user=user,
        ip=ip,
        app="DeliveryPortal",
        result=result,
    )


def detect(events):
    unique = {e["id"]: e for e in events}
    rows = sorted(unique.values(), key=lambda e: (e["time"], e["id"]))
    alerts = []
    for success in rows:
        if success["result"] != "Success":
            continue
        failures = [
            e
            for e in rows
            if e["result"] == "Failed"
            and all(e[k] == success[k] for k in ("user", "ip", "app"))
            and success["time"] - timedelta(minutes=5)
            <= e["time"]
            < success["time"]
        ]
        if len(failures) >= 3:
            alerts.append(
                dict(
                    alert_id="rule1:" + success["id"],
                    user=success["user"],
                    ip=success["ip"],
                    application=success["app"],
                    failures=len(failures),
                    success_time=success["time"].isoformat(),
                    evidence_ids=[e["id"] for e in failures]
                    + [success["id"]],
                    disposition="Needs analyst review",
                )
            )
    return alerts


pattern = [
    event("f1", 0, "Failed"),
    event("f2", 60, "Failed"),
    event("f3", 120, "Failed"),
    event("s1", 180, "Success"),
]
tests = {
    "Failures then success": (pattern, 1),
```

```
    "One typo then success": (
        [event("a", 0, "Failed"), event("b", 60, "Success")],
        0,
    ),
    "Failures without success": (pattern[:3], 0),
    "Success before failures": (
        [event("early", -60, "Success")] + pattern[:3],
        0,
    ),
    "Failures outside five minutes": (
        pattern[:3] + [event("late", 600, "Success")],
        0,
    ),
    "Duplicates must not inflate count": (
        [pattern[0]] * 3 + [pattern[3]],
        0,
    ),
    "Exactly five-minute boundary": (
        [
            event("a", 0, "Failed"),
            event("b", 60, "Failed"),
            event("c", 120, "Failed"),
            event("d", 300, "Success"),
        ],
        1,
    ),
    "Equal timestamps not proven before": (
        [
            event("a", 0, "Failed"),
            event("b", 0, "Failed"),
            event("c", 0, "Failed"),
            event("d", 0, "Success"),
        ],
        0,
    ),
    "Other IP must not be combined": (
        pattern[:3] + [event("other", 180, "Success", ip="198.51.100.20")],
        0,
    ),
    "Legitimate repeated typos, same pattern": (pattern, 1),
}
print("Detector ready. Next cell runs its tests.")
```

Python Cell 2 – Run all ten checks

Paste the entire following block into one code cell. Continue on the next page if the code spans pages. Run only after earlier cells have completed.

```
for label, (rows, expected) in tests.items():
    actual = len(detect(rows))
    assert actual == expected, (label, actual, expected)
    print(f"PASS | {label}: {actual} alert(s)")
print("10 checks passed. A matching pattern still needs investigation.")
```

Python Cell 3 – Replay events and write an alert

Paste the entire following block into one code cell. Continue on the next page if the code spans pages. Run only after earlier cells have completed.

```
incoming = queue.Queue()
emitted, seen, history = [], set(), []


def alert_worker():
    while True:
        item = incoming.get()
        try:
            if item is None:
                return
            history.append(item)
            for alert in detect(history):
                if alert["alert_id"] not in seen:
                    seen.add(alert["alert_id"])
                    emitted.append(alert)
                    print(
                        "AUTOMATIC ALERT:",
                        alert["user"],
                        "-",
                        alert["failures"],
                        "failures before success",
                    )
        finally:
            incoming.task_done()


worker = threading.Thread(target=alert_worker, daemon=True)
worker.start()
for row in pattern + [pattern[-1]]:
    incoming.put(row)
    time.sleep(0.15)
incoming.put(None)
worker.join(timeout=5)
assert not worker.is_alive(), "Worker did not stop"
assert len(emitted) == 1
Path("project1_alerts.jsonl").write_text(
    "\n".join(json.dumps(a) for a in emitted) + "\n"
)
print("PASS: one alert, repeated delivery suppressed, worker stopped.")
print(json.dumps(emitted[0], indent=2))
```

Python Cell 4 - Start the app and verify baseline

Paste the entire following block into one code cell. Continue on the next page if the code spans pages. Run only after earlier cells have completed.

```
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.request import Request, urlopen
from urllib.error import HTTPError

if "server" in globals():
    server.shutdown()
    server.server_close()
OLD_PASSWORD = "Lab-only-before-482!"
NEW_PASSWORD = "Lab-only-after-729!"
salt = secrets.token_bytes(16)


def password_hash(value):
    return hashlib.pbkdf2_hmac("sha256", value.encode(), salt, 100000)


account = {
    "user": "alex",
    "enabled": True,
    "password_hash": password_hash(OLD_PASSWORD),
}
sessions, audit = {}, []


def record(action, result):
    audit.append(
        {
            "time": datetime.now(timezone.utc).isoformat(),
            "action": action,
            "result": result,
        }
    )


class Handler(BaseHTTPRequestHandler):
    def log_message(self, *args):
        pass

    def reply(self, status, body):
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps(body).encode())

    def do_POST(self):
        if self.path != "/login":
            return self.reply(404, {"error": "unknown route"})
        data = json.loads(
            self.rfile.read(int(self.headers.get("Content-Length", 0)))
        )
        if not account["enabled"]:
            record("login", "blocked-disabled")
            return self.reply(403, {"error": "account disabled"})
        if data.get("user") != "alex" or not hmac.compare_digest(
            password_hash(data.get("password", "")),
            account["password_hash"],
        ):
            record("login", "bad-credentials")
            return self.reply(401, {"error": "bad credentials"})
```

```
            token = secrets.token_urlsafe(24)
            sessions[token] = "alex"
            record("login", "success")
            self.reply(200, {"token": token})

        def do_GET(self):
            if self.path != "/order":
                return self.reply(404, {"error": "unknown route"})
            token = self.headers.get("Authorization", "").removeprefix(
                "Bearer "
            )
            if token not in sessions:
                record("view-order", "rejected-session")
                return self.reply(401, {"error": "invalid session"})
            if not account["enabled"]:
                record("view-order", "blocked-disabled")
                return self.reply(403, {"error": "account disabled"})
            record("view-order", "success")
            self.reply(
                200,
                {"order": "TRAINING-1042", "contents": "fictional test order"},
            )


    server = ThreadingHTTPServer(("127.0.0.1", 0), Handler)
    threading.Thread(target=server.serve_forever, daemon=True).start()
    base_url = "http://127.0.0.1:" + str(server.server_port)


    def request(path, payload=None, token=None):
        headers = {"Content-Type": "application/json"}
        if token:
            headers["Authorization"] = "Bearer " + token
        req = Request(
            base_url + path,
            data=json.dumps(payload).encode() if payload else None,
            headers=headers,
        )
        try:
            with urlopen(req, timeout=3) as response:
                return response.status, json.loads(response.read())
        except HTTPError as err:
            return err.code, json.loads(err.read())


    status, body = request("/login", {"user": "alex", "password": OLD_PASSWORD})
    assert status == 200
    old_token = body["token"]
    assert request("/order", token=old_token)[0] == 200
    print("BEFORE containment: login 200; protected order access 200.")
```

Python Cell 5 – Revoke sessions and disable the account

Paste the entire following block into one code cell. Continue on the next page if the code spans pages. Run only after earlier cells have completed.

```
sessions.clear()
record("admin-revoke-sessions", "completed")
account["enabled"] = False
record("admin-disable-account", "completed")
old_access = request("/order", token=old_token)[0]
fresh_login = request("/login", {"user": "alex", "password": OLD_PASSWORD})[
    0
]
assert old_access == 401 and fresh_login == 403
print("CONTAINED: old session", old_access, "; fresh login", fresh_login)
print(
    "Verified by rejected requests, not merely by an admin success message."
)
```

Python Cell 6 – Recover verify export and stop

Paste the entire following block into one code cell. Continue on the next page if the code spans pages. Run only after earlier cells have completed.

```
account["password_hash"] = password_hash(NEW_PASSWORD)
account["enabled"] = True
record("admin-reset-and-enable", "completed")
assert (
    request("/login", {"user": "alex", "password": OLD_PASSWORD})[0] == 401
)
assert request("/order", token=old_token)[0] == 401
status, body = request("/login", {"user": "alex", "password": NEW_PASSWORD})
assert status == 200 and request("/order", token=body["token"])[0] == 200
print(
    "RECOVERY PASS: old password 401; old token 401; new login and order access 200."
)
Path("project1_response_audit.json").write_text(json.dumps(audit, indent=2))
server.shutdown()
server.server_close()
print("Training server stopped. No enterprise accounts were changed.")
```

Appendix B - Troubleshooting

Problem	Fix
Azure says table not found	Select the database where you created the table. Verify spelling and run the schema-creation step only if missing.
LoginLogs count is above six	The ingestion block may have run twice. Use a new empty training database and ingest once. Do not delete unrelated data.
No rows in the time filter	Use the fixed 23 September 2026 UTC times; a relative current-time filter excludes the fixtures.
Management command errors	Run each .create or .ingest block separately. Do not paste all of azure-queries.kql and run it as one command.
Jira link missing	Open the item itself; find Linked work items or Link work item in its menu. Check site permissions if the control is unavailable.
Wrong Jira relationship	Remove only the erroneous relation and save relates to. A subtask is a separate item, not an existing-item link.
Colab NameError	Run the earlier setup cells in order. A runtime restart clears Python variables.
IndentationError or SyntaxError	Copy the entire cell without PDF headers. Prefer the matching plain .py file if PDF copying damaged spacing.
Connection refused in response cells	The server stopped or Cell 4 did not run. Rerun Cell 4, then 5 and 6 in order.
Audit files disappeared	Runtime storage is temporary. Rerun the exercise and download both files before disconnecting.

If a result differs, record the actual error and affected step. Do not replace it with the expected result or describe the step as completed until the difference is understood.

Appendix C - Evidence and references

The figures are excerpts from uploaded screenshots of this lab. A cropped figure is a teaching view of the original screen; it is not a new run. Code blocks and expected outputs are labelled separately. No fabricated screenshots are included.

Figure 1: 01-Screenshot-2026-09-23-at-2.12.52-PM.png
Creation form before submission. It still shows default names; replace them with your chosen lab names.

Figure 2: 01-Screenshot-2026-09-23-at-2.19.56-PM.png
A one-row result confirms the query ran.

Figure 3: 01-Screenshot-2026-09-23-at-2.27.57-PM.png
The empty table shows its five columns and no rows.

Figure 4: 01-Screenshot-2026-09-23-at-2.38.40-PM.png
Historical ingestion confirmation, with HasErrors = false. That screenshot loaded one row; the consolidated block above loads all six together.

Figure 5: 01-Screenshot-2026-09-23-at-3.17.01-PM.png
Filtering and counting the failed events.

Figure 6: 01-Screenshot-2026-09-23-at-3.17.01-PM.png
The recorded failure count is three.

Figure 7: 01-Screenshot-2026-09-23-at-5.37.01-PM.png
The space name and key fields before completion.

Figure 8: 01-Screenshot-2026-09-23-at-5.50.28-PM.png
The empty SOC board before case creation.

Figure 9: 01-Screenshot-2026-09-24-at-1.59.43-AM.png
The saved relates-to relationship. This screenshot shows an intermediate case state.

Figure 10: 01-Screenshot-2026-09-24-at-2.30.24-AM.png
Activity, Comments and the comment entry area.

Figure 11: 01-Screenshot-2026-09-24-at-3.39.25-AM.png
The original notebook after the first setup cell ran. The companion uses the same logic with clearer formatting.

Figure 12: 01-Screenshot-2026-09-24-at-3.46.24-AM.png
One automatic alert; repeated delivery suppressed; worker stopped.

Figure 13: 02-Screenshot-2026-09-24-at-3.46.37-AM.png
Containment code and the rejected requests.

Figure 14: 03-Screenshot-2026-09-24-at-3.46.41-AM.png
Recovery verification and server shutdown.

Official references checked for this walkthrough

[1] Microsoft - free cluster setup
https://learn.microsoft.com/en-us/azure/data-explorer/start-for-free-web-ui

[2] Google - Colab FAQ
https://research.google.com/colaboratory/faq.html

[3] Atlassian - Jira Free
https://www.atlassian.com/software/jira/free

[4] Atlassian - create work items
https://support.atlassian.com/jira-software-cloud/docs/create-a-work-item-and-a-subtask/

[5] Atlassian - link work items
https://support.atlassian.com/jira-software-cloud/docs/link-issues/

[6] Microsoft - sign-in activity details
https://learn.microsoft.com/en-us/entra/identity/monitoring-health/concept-sign-in-log-activity-details

Interface labels and free-tier availability may change. Screenshots preserve the layout used during the original lab; use the named action and official reference when your layout differs.